

NLU course project: Language Modeling (Lab 4)

Kevin Shimaj (mat. 267432)

University of Trento

kevin.shimaj@studenti.unitn.it

1. Introduction

This report covers the two language modeling tasks on the Penn Treebank (PTB) corpus [1]. In part 1.A a GPT-2 style decoder [2] is trained from scratch, starting from the tiny lab baseline and growing it one hyperparameter at a time; a change is kept only when the validation perplexity (PPL) improves, and every run is reported, including the rejected ones. The final model (256 hidden size, 8 heads, 6 layers, dropout 0.2) reaches a test PPL of 30.39, well below the 250 target; weight tying was also implemented and evaluated, but rejected after four controlled runs. In part 1.B the pretrained GPT-2 is adapted to PTB by training only hand-written LoRA adapters [3] on the attention projections, with the constraint of beating the part 1.A result. The adapted model reaches a test PPL of 20.61, below both the target and the part 1.A score, while updating only 0.35% of the weights.

2. Implementation details

Part 1.A. All runs share the lab protocol: GPT-2 BPE tokenizer (pad token set to eos), next-token prediction with inputs and labels shifted by one, token-weighted cross entropy, AdamW, batch size 8 for training and 16 for evaluation, early stopping with patience 3 on validation PPL. PPL is the exponential of the token-weighted mean loss, and the reported test score is always the one of the best validation checkpoint. The search had three phases. (i) A learning rate sweep on the tiny baseline over $\{5e-4, 1e-3, 2e-3, 5e-3\}$, a log-spaced grid centered on the stable AdamW range; the lab value 0.1 is unstable and stalls at high PPL. (ii) Architecture growth, one knob at a time: d_{model} from 20 to 256 (384 hurt; the embedding table dominates the parameter count, so width saturates first), heads from 1 to 8 (almost free, since they only reshape the attention projections), layers from 1 to 6 (gains shrink geometrically), and feed-forward size from 20 to 1024, i.e. $4 \times d_{\text{model}}$ as in the standard Transformer [4], which gave the largest single improvement. (iii) A second sweep on the final architecture, where $2e-3$ won: this model overfits PTB in a few epochs, and under early stopping the gradient noise of a higher rate works as an implicit regularizer. The value $5e-3$ was not rerun because it was already the worst point of the first sweep and deeper networks are less tolerant to large steps. (iv) Dropout [5] at the four lab locations: after the summed embeddings, on the attention weights after the softmax, after the attention output projection, and in the feed-forward block after the activation and after the last linear layer. It pays here precisely because the final model overfits PTB within a few epochs: $p=0.1$ gains 2 validation PPL and $p=0.2$ another 1.3, with training surviving more epochs before early stopping (22 vs 10). (v) Weight tying [6] shares the embedding matrix with the output head (-12.9M parameters). It interacts badly with the lab initialization: the uniform re-init of the head silently flattens the tied embeddings to near zero and training stalls. Re-initializing token and positional embeddings

at the GPT-2 scale, $\mathcal{N}(0, 0.02)$, restores a healthy start, but convergence then crawls towards a plateau roughly 15 PPL above the untied model — identically at both dropout levels, so the cost is the shared input/output geometry itself and not an excess of regularization. The tie was therefore reverted.

Part 1.B. The whole pretrained model is frozen and each attention block gets a low-rank adapter on the Q, K, V projections: the output becomes $Wx + B(A(x))\alpha/r$, with A initialized from a normal distribution and B at zero, so training starts exactly at the pretrained weights and only the adapters are updated. The search followed the idea behind LoRA, that the weight update needed to adapt a large model has a low intrinsic rank. It started from a small rank of 8 with the scale α set equal to the rank, which is the neutral choice. This first setting already passed both targets, so the effort moved to the learning rate: $1e-3$ improved over $5e-4$, which fits the fact that updating only a small fraction of the weights tolerates a larger step. Higher ranks were not explored, since a low intrinsic rank means that extra capacity is expected to help little while doubling or quadrupling the adapter size.

3. Results

Table 1 lists every part 1.A run in chronological order. Figure 1 shows the chain of accepted steps: capacity dominates at the start (width, then depth), the feed-forward expansion gives the largest single architectural drop (-3.4 validation PPL), consistent with the fact that a 20-unit MLP was a strong bottleneck for 256-dimensional states, and the two dropout steps close the ladder with a further -3.3 . The four weight-tying runs (19–22 in the table) were all rejected: two stalled because of the initialization interaction described above and two converged healthily but towards a far worse plateau, so they were aborted once the trajectory was clearly dominated (their dev column reports the last value before the manual stop). Figure 2 compares the two learning rate sweeps: the optimum moves from $1e-3$ on the tiny model to $2e-3$ on the final one, while the low end ($5e-4$) gets clearly worse, which supports the regularization reading given above. Note that test PPL is consistently below validation PPL on PTB; this is a property of the split, not a bug. Table 2 reports part 1.B. Both settings clear the targets by a wide margin, and the final adapter reaches a test PPL of 20.61 against the 30.39 of part 1.A while updating only 0.35% of the parameters. This large gap is expected: part 1.B starts from a model already pretrained on a much larger corpus and only shifts it towards PTB, whereas part 1.A learns the language from scratch on the small PTB training set.

Table 1: *Part 1.A, full incremental log. Config is $d_{\text{model}}/\text{heads}/\text{layers}/\text{ff}$. Decision: K = kept, R = rejected (reverted). In runs 20–21 “init .02” is the $\mathcal{N}(0, 0.02)$ re-init of the tied token (and positional) embeddings. $\dagger$: run aborted once clearly dominated; value is the validation PPL at the manual stop (after 8, 1, 25 and 9 epochs respectively).*

#	Step	Config	lr	Dev	Test	D
1	baseline (lab)	20/1/1/20	0.1	95.45	82.97	–
2	lr sweep	20/1/1/20	1e−3	51.93	45.28	K
3	lr sweep	20/1/1/20	2e−3	52.32	46.07	R
4	lr sweep	20/1/1/20	5e−3	53.62	47.08	R
5	lr sweep	20/1/1/20	5e−4	52.40	45.44	R
6	d_{model}	128/1/1/20	1e−3	45.68	40.70	K
7	d_{model}	256/1/1/20	1e−3	45.53	40.74	K
8	d_{model}	384/1/1/20	1e−3	45.73	40.65	R
9	heads	256/4/1/20	1e−3	43.55	39.13	K
10	heads	256/8/1/20	1e−3	43.47	39.11	K
11	layers	256/8/2/20	1e−3	42.25	38.05	K
12	layers	256/8/4/20	1e−3	41.34	37.21	K
13	layers	256/8/6/20	1e−3	40.82	36.90	K
14	ff ($4 \times d$)	256/8/6/1024	1e−3	37.39	33.92	K
15	lr re-tune	256/8/6/1024	5e−4	39.12	35.14	R
16	lr re-tune	256/8/6/1024	2e−3	37.09	33.68	K
17	dropout $p=0.1$	256/8/6/1024	2e−3	35.07	31.65	K
18	dropout $p=0.2$	256/8/6/1024	2e−3	33.79	30.39	K
19	tie, lab init	256/8/6/1024	2e−3	151 $\dagger$	–	R
20	tie, init .02 (tok)	256/8/6/1024	2e−3	241 $\dagger$	–	R
21	tie, init .02 (tok+pos)	256/8/6/1024	2e−3	48.4 $\dagger$	–	R
22	tie + drop $p=0.1$	256/8/6/1024	2e−3	57.5 $\dagger$	–	R

Table 2: *Part 1.B, LoRA fine-tuning of pretrained GPT-2 on PTB. Reference to beat: test PPL 30.39 (part 1.A). Trainable is the share of updated weights. Decision: K = kept, R = rejected.*

#	rank	α	lr	Trainable	Dev	Test	D
1	8	8	5e−4	0.35%	23.36	21.11	R
2	8	8	1e−3	0.35%	22.83	20.61	K

4. References

- [1] M. P. Marcus, B. Santorini, and M. A. Marcinkiewicz, “Building a large annotated corpus of English: The Penn Treebank,” *Computational Linguistics*, vol. 19, no. 2, pp. 313–330, 1993.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Technical Report*, 2019.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [6] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *Proceedings of EACL*, 2017.

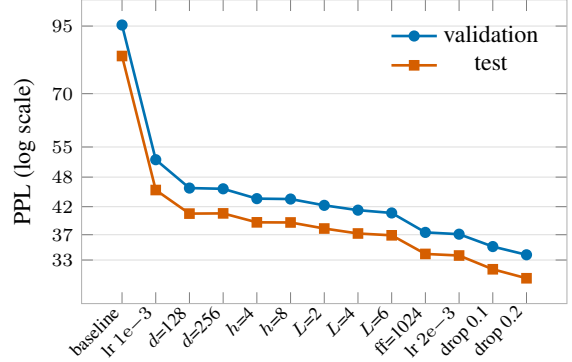


Figure 1: *Part 1.A: perplexity along the chain of accepted steps. Each point is the run that was kept at that stage of the search (rejected runs are in Table 1).*

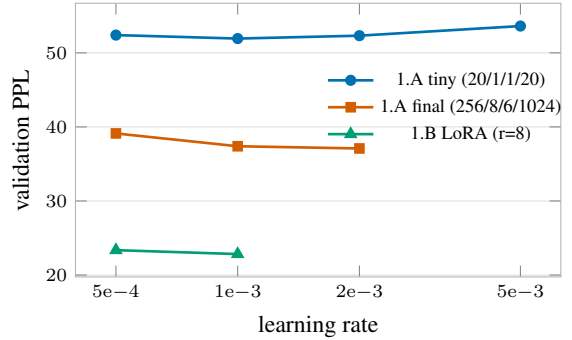


Figure 2: *Learning rate sweeps. For part 1.A the optimum moves from 1e−3 on the tiny model to 2e−3 on the final one, since the larger model overfits quickly and a higher rate with early stopping generalizes better. Part 1.B, the LoRA fine-tuning, sits much lower and prefers 1e−3, as adapting only a small fraction of the weights tolerates a larger step.*